

Rezumat Cursuri 11 & 11b — Căutare Neinformată, Căutare Locală, Algoritmi Evolutivi, PSO, ACO

Surse: 11_search_uninformed.pdf + 11_localSearch_EA.pdf + 11_localSearch_EA_suplim.pdf + 11_local_search_PSO_ACO.pdf + fișiere .md 9_1 → 9_7 + Criterii_de_oprire.md **UBB Informatică Anul 2 — Inteligență Artificială**

PARTEA I — CĂUTARE NEINFORMATĂ (Blind Search)

1. Rezolvarea Problemelor prin Căutare

1.1 Definiția problemei de căutare

O problemă de căutare se definește prin:

- **Spațiu de stări** — toate configurațiile posibile
- **Stare inițială** — punctul de start
- **Stare finală (obiectiv)** — ce căutăm
- **Drumuri** — succesiuni de stări
- **Operatori (funcții succesor)** — transformă o stare în alta
- **Funcție de cost** — evaluează calitatea unui drum
- **Funcție obiectiv** — verifică dacă s-a ajuns la starea finală

Exemplu — 8-puzzle: Starea = configurația tablei cu 8 piese. Operatori = mutarea piesei goale sus/jos/stânga/dreapta. Obiectiv = piesele aranjate în ordine.

1.2 Tipologia strategiilor de căutare

Strategii de Căutare (SC)

- └ SCnI — Neinformate (oarbe/blind)
 - └ Structuri liniare: Căutare liniară, Căutare binară
 - └ Structuri ne-liniare: BFS, UCS, DFS, DLS, IDS, BDS
- └ SCI — Informate (euristice)
 - └ Globale: Best-first, Greedy, A*
 - └ Locale: Hill Climbing, Simulated Annealing, Tabu Search, Algoritmi Evolutivi, PSO, ACO

Notații comune:

- b = factorul de ramificare (nr. de copii ai unui nod)
- d = adâncimea soluției (nr. de pași până la obiectiv)
- d_{max} = adâncimea maximă a arborelui explorat

2. BFS — Breadth-First Search (Căutare în Lățime)

2.1 Principiu

Explorează nodurile **nivel cu nivel** — mai întâi toate nodurile de la adâncimea d , apoi toate de la $d + 1$.

Structura de date: Coadă **FIFO** (First In, First Out)

2.2 Algoritm

```
BFS(elem):
    toVisit = {start} // coadă FIFO
    visited = {}
    while toVisit != {} și !found:
        node = pop(toVisit)
        visited = visited u {node}
        if node == elem: found = true
        else: toVisit = toVisit u {copii nevizitați ai lui node}
```

2.3 Exemplu pas cu pas

Graf:



Iterație	Vizitate	De vizitat
Start	{}	{A}
1	{A}	{B, C, D}
2	{A, B}	{C, D, E, F}
3	{A, B, C}	{D, E, F, G}
4	{A, B, C, D}	{E, F, G, H, I, J}
5-10	...	...
Final	{A,B,C,D,E,F,G,H,I,J,K}	{}

Ordinea vizitării: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow I \rightarrow J \rightarrow K$

2.4 Analiză

Proprietate	Valoare	Explicație
Complexitate temporală	$O(b^d)$	Toate nodurile până la adâncimea d
Complexitate spațială	$O(b^d)$	Toate nodurile stocate
Completitudine	Da	Dacă soluția există, BFS o găsește
Optimalitate	Nu	Găsește cel mai scurt drum (nr. pași), NU cel mai ieftin

Atenție: BFS găsește soluția cu **cei mai puțini pași**, nu neapărat cea mai ieftină!

3. DFS — Depth-First Search (Căutare în Adâncime)

3.1 Principiu

Explorează **cât mai adânc posibil** înainte de a se întoarce.

Structura de date: Stivă **LIFO** (Last In, First Out)

3.2 Exemplu pas cu pas

Pe același graf:

Ordinea vizitării: $A \rightarrow B \rightarrow E \rightarrow F \rightarrow C \rightarrow G \rightarrow D \rightarrow H \rightarrow I \rightarrow J \rightarrow K$

3.3 Analiză

Proprietate	Valoare
Complexitate temporală	$O(b^{d_{max}})$
Complexitate spațială	$O(b \cdot d_{max})$
Completitudine	Nu (poate cădea în bucle infinite)
Optimalitate	Nu

4. UCS — Uniform Cost Search (Căutare de Cost Uniform)

4.1 Principiu

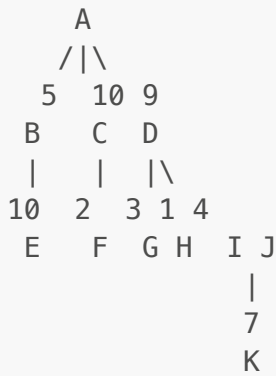
Ca BFS, dar expandează întotdeauna **nodul cu costul cumulativ cel mai mic**.

Structura de date: Coadă FIFO **sortată** după cost total

Echivalent cu: Algoritmul Dijkstra

4.2 Exemplu pas cu pas

Graf cu costuri:



Vizitate	De vizitat (cost)
{}	{A(0)}
{A(0)}	{B(5), C(10), D(9)}
{A,B(5)}	{C(9)←, D(9), F(8), E(15)}
{A,B,F(8)}	{C(9), D(9), E(15)}
...	...

Notă: UCS recalculează costurile și poate actualiza prioritățile.

4.3 Analiză

Proprietate	Valoare
Completitudine	Da
Optimalitate	Da (găsește drumul de cost minim)
Complexitate	$O(b^d)$

5. DLS — Depth-Limited Search & IDS — Iterative Deepening

DLS (Căutare în Adâncime Limitată)

- DFS cu o limită de adâncime d_{lim}
- Completitudine: **Da** dacă $d_{lim} > d$
- Optimalitate: **Nu**

IDS (Iterative Deepening Depth-First Search)

- Repetă DLS cu limite crescătoare: 0, 1, 2, 3, ...
- Combină avantajele BFS (completitudine, optimalitate) și DFS (memorie redusă)
- Completitudine: **Da**; Complexitate spațială: $O(b \cdot d)$

6. Căutare Bidirecțională (BDS)

Pornește simultan din **starea inițială** și **starea finală**, se întâlnesc la mijloc.

- Complexitate temporală/spațială: $O(b^{d/2})$ — mult mai eficientă!
- Completitudine și optimalitate: **Da**

7. Tabel Comparativ Căutare Neinformată

Metodă	Complexitate T	Complexitate S	Completitudine	Optimalitate
BFS	$O(b^d)$	$O(b^d)$	Da	Nu*
UCS	$O(b^d)$	$O(b^d)$	Da	Da
DFS	$O(b^{d_{max}})$	$O(b \cdot d_{max})$	Nu	Nu
DLS	$O(b^{d_{lim}})$	$O(b \cdot d_{lim})$	Da (dacă $d_{lim} > d$)	Nu
IDS	$O(b^d)$	$O(b \cdot d)$	Da	Da
BDS	$O(b^{d/2})$	$O(b^{d/2})$	Da	Da

*BFS e optim pentru costuri uniforme (toți pașii costă la fel).

8. Căutare Informată — Best First Search & A*

8.1 Notății SCI

- $f(n)$ = funcție de evaluare totală
- $g(n)$ = costul real de la start până la nodul n
- $h(n)$ = euristica — estimarea costului de la n la obiectiv
- $f(n) = g(n) + h(n)$

```

start ---g(n)----> n ---h(n)----> obiectiv
      real      estimat
  
```

8.2 Greedy Best-First Search

- $f(n) = h(n)$ — merge direct spre obiectiv ignorând costul parcurs
- **Rapid** dar **nu optim**

8.3 A*

- $f(n) = g(n) + h(n)$ — combină costul real + estimarea
- **Optim și complet** dacă $h(n)$ este admisibilă ($h(n) \leq$ costul real)
- Combină avantajele UCS (optimalitate) + Greedy (viteză)

PARTEA a II-a — CĂUTARE LOCALĂ

9. Căutare Locală — Introducere

Definiție: Se pornește dintr-o singură stare și se explorează **vecinii** acesteia, modificând treptat soluția curentă.

Spre deosebire de căutarea globală: Nu construiește un arbore — lucrează direct cu soluția, fără a memora drumul parcurs.

Tipologie:

- **Simple** (o singură stare reținută): Hill Climbing, Simulated Annealing, Tabu Search
- **În fascicol / beam** (populație de stări): Algoritmi Evolutivi, PSO, ACO

10. Hill Climbing (HC)

10.1 Principiu

"Urcarea unui munte în ceață și cu amnezie" — mergi mereu în direcția **îmbunătățirii**, oprești când nu mai există vecin mai bun.

Criteriu de acceptare: Cel mai bun vecin al soluției curente, dacă e mai bun decât soluția curentă.

10.2 Algoritm

```
HC():
    x = starea_initiala
    x* = x
    cât timp nu criteriu_oprire:
        generează toți vecinii N ai lui x
        s = cel mai bun din N
        dacă f(s) > f(x): x = s
        altfel: stop
    returnează x*
```

10.3 Exemplu — Turnuri din piese geometrice

Problemă: Ai n piese de diferite lungimi pe 3 stive, trebuie reordonate în ordine crescătoare pe stiva 1.

Funcția de evaluare:

- f_1 = suma înălțimilor pieselor corect amplasate (maximizare)
- f_2 = suma înălțimilor pieselor incorect amplasate (minimizare)
- $f = f_1 - f_2$ (maximizare)

Iterația 1: Stare inițială $s_1 = ((5,1), (1,1), (2,1), (3,1), (4,1)) \rightarrow f(s_1) = 0 - 10 = -10$ Singurul vecin: piesa 5 pe stiva 2 $\rightarrow f(s_2) = -6 > -10 \rightarrow x = s_2$

Iterația 2: $x = ((1,1), (2,1), (3,1), (4,1), (5,2)), f(x) = -6$ Doi vecini posibili:

- piesa 1 pe stiva 2 $\rightarrow f = -5 > -6 \checkmark$

- piesa 1 pe stiva 3 $\rightarrow f = -4 > -6 \checkmark$ (mai bun) $\rightarrow x = s_4 = ((2, 1), (3, 1), (4, 1), (5, 2), (1, 3))$

Algoritmul continuă până nu există vecin mai bun.

10.4 Problema optimelor locale

HC converge la **optimul local**, nu neapărat global!

Situații problematice:

- **Optim local** — niciun vecin nu e mai bun, dar nu e soluția globală
- **Platou** — toți vecinii au același scor (zona plată)
- **Creastă** — ar trebui să faci un pas mai slab pentru a progresa

Variante HC:

- **HC stocastic** — alege aleator un vecin mai bun (nu neapărat cel mai bun)
- **HC cu prima opțiune** — primul vecin mai bun găsit
- **HC cu repornire aleatoare (beam search)** — repornire din stare aleatoare când nu progresează

10.5 Analiză

Proprietate	Valoare
Convergență	Spre optimul local
Completitudine	Nu
Optimalitate	Nu
Avantaj	Simplu, rapid, fără memorie
Dezavantaj	Se blochează în optime locale

11. Simulated Annealing (SA)

11.1 Principiu

Inspirat din **fizica proceselor de răcire** (Metropolis 1953, Kirkpatrick 1982).

Idee cheie: Spre deosebire de HC, SA acceptă uneori și mutări **mai slabe**, cu o probabilitate care scade în timp.

Probabilitatea de acceptare a unui vecin mai slab:

$$p = e^{\Delta E/T}$$

Unde:

- ΔE = diferența de calitate (negativă dacă noul stat e mai slab)
- T = temperatura — parametru care scade gradual

11.2 Comportament

- T mare $\rightarrow$ mutările slabe sunt acceptate frecvent (explorare)
- $T \rightarrow 0 \rightarrow$ SA devine Hill Climbing (exploatare)
- $T \rightarrow \infty \rightarrow$ toate mutările acceptate (aleatoare)

Criteriu de acceptare: Un vecin aleator mai bun $\rightarrow$ acceptat mereu. Mai slab $\rightarrow$ acceptat cu probabilitatea $p = e^{\Delta E/T}$.

11.3 Scăderea temperaturii (cooling schedule)

Temperatura scade treptat: $T_{t+1} = \alpha \cdot T_t$, unde $\alpha \in (0.8, 0.99)$.

Dacă temperatura scade prea repede $\rightarrow$ se comportă ca HC (se blochează în optime locale). **Dacă temperatura scade prea lent** $\rightarrow$ converge spre optim global dar e lent.

11.4 Exemplu — Problema celor 8 regine

Problemă: Amplasarea a 8 regine pe o tablă de șah astfel încât să nu se atace reciproc.

Stare: Vector de 8 elemente — poziția pe coloană a fiecărei regine.

Ex: $[1, 3, 5, 7, 2, 4, 6, 8]$ $\rightarrow$ regina 1 pe coloana 1, regina 2 pe coloana 3, etc.

Funcție de evaluare: Numărul de perechi de regine care se atacă (minimizare).

SA poate "sări" peste configurații mai proaste pentru a evita optimele locale.

12. Căutare Tabu

12.1 Principiu

Extinde HC prin menținerea unei **liste tabu** — o memorie a soluțiilor recent vizitate. Evită revenirea în aceleași stări.

Criteriu de acceptare: Cel mai bun vecin care **nu se află în lista tabu**.

12.2 Mecanism

- Lista tabu are o **lungime fixă** (parametru)
- Când lista e plină, se elimină cea mai veche intrare
- **Criteriu de aspirare:** Chiar dacă o soluție e în lista tabu, poate fi acceptată dacă e mai bună decât cea mai bună soluție găsită vreodată

PARTEA a III-a — ALGORITMI EVOLUTIVI

13. Algoritmi Evolutivi — Principii Generale

13.1 Ce sunt?

Algoritmi de **optimizare** inspirați din **evoluția biologică** (selecție naturală, Darwin). Lucrează cu o **populație** de soluții candidate.

13.2 Analogia biologică

Biologie	Informatică
Individ	Soluție candidat
Cromozom / Genotip	Reprezentarea internă (codul soluției)
Fenotip	Interpretarea externă (soluția evaluată)
Genă	Element individual al cromozomului
Locus	Poziția unei gene
Alelă	Valoarea pe care o poate lua o genă
Fitness	Calitatea soluției ($f(x)$)
Selecție	Favorizarea soluțiilor mai bune pentru reproducere
Încrucișare	Combinarea a doi cromozomi → copii
Mutație	Modificare aleatoare mică

13.3 Schema generală

```

Inițializează populația  $P(0)$  cu  $N$  indivizi (aleator)
Evaluează fiecare individ: calculează  $f(x)$  pentru fiecare  $x$  din  $P(0)$ 
 $t = 0$ 
Cât timp nu criteriu_oprire:
     $t = t + 1$ 
    Selectează părinți din  $P(t-1)$ 
    Aplică crossover → generează descendenți
    Aplică mutație pe descendenți
    Evaluează descendenții
    Formează  $P(t)$  din descendenți (+ elitism opțional)
Returnează cel mai bun individ din  $P(t)$ 

```

14. Algoritmi Genetici (AG)

14.1 Caracteristici

Sunt o **subclasă** de algoritmi evolutivi care folosesc:

- **Codificare binară** (tipic) sau vectori de numere
- **Crossover** ca operator principal
- **Selecție** bazată pe fitness

14.2 Reprezentarea soluțiilor

Fiecare soluție = **cromozom** = vector de gene.

Tipuri de reprezentare:

- **Binară:** $x = [1, 0, 1, 1, 0, 1]$ — fiecare genă e 0 sau 1
- **Întreagă:** $x = [2, 5, 1, 4, 3]$ — permutări (ex: TSP)
- **Reală:** $x = [2.3, -1.5, 0.7]$ — optimizare continuă

Important: Cromozomii NU sunt limitați la valori 0 și 1! Depinde de problema reprezentată.

14.3 Exemplu complet — Maximizarea $f(x) = x^2$

Problemă: Găsim valoarea maximă a $f(x) = x^2$ pentru $x \in \{0, 1, \dots, 31\}$.

Reprezentare: Șiruri binare de lungime 5. Ex: $(10101)_2 = 21 \rightarrow f(21) = 441$.

Populație inițială (4 cromozomi):

#	Cromozom	x (valoare)	Fitness $f = x^2$
1	01101	13	169
2	11000	24	576
3	01000	8	64
4	10011	19	361

Total fitness: $169 + 576 + 64 + 361 = 1170$

14.4 Selecția (Roulette Wheel)

Probabilitate de selecție proporțională cu fitness-ul:

$$p_i = \frac{f(x_i)}{\sum_{j=1}^N f(x_j)}$$

#	Fitness	Probabilitate	Interval pe ruletă
1	169	$169/1170 = 14.4\%$	$[0, 0.144)$
2	576	$576/1170 = 49.2\%$	$[0.144, 0.636)$
3	64	$64/1170 = 5.5\%$	$[0.636, 0.691)$
4	361	$361/1170 = 30.9\%$	$[0.691, 1.0)$

Cromozomii cu fitness mai mare au „sectoare” mai mari pe ruletă → șanse mai mari de selecție.

14.5 Crossover (Încrucișare)

One-point crossover (cu un punct de tăietură)

Se alege aleator un punct k și se schimbă genele după acel punct:

Părinți: cromozom 2 și cromozom 4 cu punct la poziția 2:

Cromozom 2: 11|000 → Copil 1: 11|011
Cromozom 4: 10|011 → Copil 2: 10|000

	Cromozom	x	Fitness
Copil 1	11011	27	729
Copil 2	10000	16	256

Copilul 1 (729) e mai bun decât ambii părinți (576 și 361)!

Two-point crossover (cu două puncte de tăietură)

Cromozom 1: 01|101|01 → Copil 1: 01|011|01
Cromozom 4: 10|011|11 → Copil 2: 10|101|11

Uniform crossover

Fiecare genă e aleasă aleator de la unul dintre cei doi părinți.

14.6 Mutația

Modifică aleator gene din cromozom pentru a introduce diversitate.

Mutație binară (inversarea unui bit):

$$x_i \leftarrow 1 - x_i$$

Exemplu: Copil 1 după crossover = 11011, se inversează poziția 3:

11011 → 11001 (bitul 3 schimbat din 1 în 0)

Valoarea devine: $x = (11001)_2 = 25$, $f = 625$

Mutație reală:

$$x_i \leftarrow x_i + \mathcal{N}(0, \sigma)$$

Probabilitatea de mutație trebuie să fie **mică** (ex: 0.01) — prea multă mutație distruge soluțiile bune!

14.7 Generational GA vs Steady-State GA

	Generational GA	Steady-State GA
Înlocuire	Toată populația	Doar câțiva indivizi
Evoluție	Pe generații complete	Incrementală
Elitism	Necesită mecanism explicit	Inerent în proces
Diversitate	Mai mare	Mai scăzută
Convergență	Mai rapidă	Mai stabilă

14.8 Selecție prin turneu

1. Se aleg aleator k indivizi din populație
2. Se selectează **cel mai bun** dintre ei
3. Procedul se repetă pentru fiecare slot de selecție

Avantaj față de ruletă: Nu depinde de scara fitness-ului (nu e vulnerabil la dominarea unui singur individ cu fitness uriaș).

14.9 Criterii de oprire

1. **Număr maxim de generații** atins (ex: 1000)
2. **Fitness satisfăcător** — cel mai bun individ depășește un prag
3. **Convergența populației** — toți indivizii au același fitness
4. **Lipsa îmbunătățirii** — fitness-ul nu s-a îmbunătățit în ultimele k generații

15. TSP — Problema Comisului Voiajor

15.1 Enunț

Un comis voiajor trebuie să viziteze n orașe **exact o singură dată** și să se întoarcă la punctul de start, cu **distanța totală minimă**.

Complexitate: $O(n!)$ — pentru $n = 20$ orașe există ~2.4 trilioane de tururi posibile.

15.2 Distanța dintre două orașe

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

15.3 Reprezentarea cromozomilor pentru TSP

Fiecare cromozom = **permutare** a orașelor. Ex: pentru $n = 5$: $[2, 3, 5, 1, 4]$

Aceasta înseamnă: pornești din 2 → 3 → 5 → 1 → 4 → înapoi la 2.

15.4 Funcția de fitness

$$f(T) = \sum_{i=1}^{n-1} d(t_i, t_{i+1}) + d(t_n, t_1)$$

Minimizăm distanța totală. Pentru selecție proporțională (maximizare):

$$\text{fitness}(T) = \frac{1}{f(T)}$$

15.5 Exemplu numeric TSP cu 4 orașe

Orașe și coordonate:

- $A = (0, 0), B = (1, 0), C = (1, 1), D = (0, 1)$

Tur 1: $[A, B, C, D]$:

$$AB + BC + CD + DA = 1 + 1 + 1 + 1 = 4$$

Tur 2: $[A, C, B, D]$:

$$AC + CB + BD + DA = \sqrt{2} + 1 + \sqrt{2} + 1 \approx 5.83$$

Tur 1 e mai bun $\rightarrow$ fitness mai mare.

15.6 Order Crossover (OX) pentru TSP

Crossover special care respectă constrângerile permutărilor (fără duplicate):

Părinți:

```
P1 = [1, 2, 3 | 4, 5, 6 | 7, 8]
P2 = [2, 4, 6 | 8, 1, 3 | 5, 7]
```

1. Se copiază segmentul $[4, 5, 6]$ din P1 în copil
2. Se completează restul în ordinea în care apar în P2 (sărimu-le pe cele deja prezente)

15.7 Operatori de mutație pentru TSP

- **Swap mutation:** Se schimbă 2 elemente din permutare
- **Inversion mutation:** Se inversează o secțiune a permutării
- **Scramble mutation:** Se rearanjează aleator o secțiune

15.8 Reprezentări pentru algoritmi genetici (din 9_4)

Tip problemă	Reprezentare cromozom	Exemplu
Optimizare binară	Vector binar	$[1, 0, 1, 1, 0]$
Optimizare reală	Vector real	$[2.3, -1.5, 0.7]$
Permutare (TSP)	Permutare de întregi	$[3, 1, 4, 2, 5]$
Planificare melodii	Vector de întregi cu valori din $\{1, \dots, n\}$	Ex: k melodii, n playlists

PARTEA a IV-a — PSO

16. PSO — Particle Swarm Optimization

16.1 Inspirație

Comportamentul colectiv al **roiurilor** (stoluri de păsări, bancuri de pești) — Kennedy & Eberhart, 1995.

Intuiție: Un stol de păsări caută zona cu cea mai multă hrană. Fiecare pasăre ține minte unde a găsit cea mai multă hrană și știe unde a găsit cea mai multă hrană vecinul ei cel mai bun.

16.2 Notății

Simbol	Semnificație
N	Numărul total de particule
D	Dimensiunea spațiului de căutare
$x_i(t)$	Poziția particulei i la momentul t (= soluție candidat)
$v_i(t)$	Viteza particulei i la momentul t
$pBest_i$	Cea mai bună poziție găsită de particula i (personal best)
$gBest$	Cea mai bună poziție globală din tot roiul (global best)
$lBest$	Cea mai bună poziție din vecinătatea locală

16.3 Algoritmul PSO

- Inițializare:
 - Fiecare particulă primește: poziție aleatoare + viteză nulă/aleatoare
 - $pBest[i] = x[i](0)$
- Evaluare:
 - Calculează fitness pentru fiecare particulă
 - Actualizează $pBest[i]$ dacă $fitness(x[i]) > fitness(pBest[i])$
- Pentru fiecare particulă:
 - Actualizează $gBest$ (cel mai bun $pBest$ din tot roiul)
 - Actualizează viteza:

$$vid = w * vid + c1 * r1 * (pBestid - xid) + c2 * r2 * (gBestd - xid)$$
 - Actualizează poziția:

$$xid = xid + vid$$
- Dacă nu criteriu_oprire → revino la pasul 2

16.4 Formula vitezei — explicată complet

$$v_{id}(t+1) = \underbrace{w \cdot v_{id}(t)}_{\text{inerție}} + \underbrace{c_1 \cdot r_1 \cdot (pBest_{id} - x_{id}(t))}_{\text{termen cognitiv}} + \underbrace{c_2 \cdot r_2 \cdot (gBest_d - x_{id}(t))}_{\text{termen social}}$$

Unde:

- $i = 1, \dots, N$ (numărul particulei)
- $d = 1, \dots, D$ (dimensiunea curentă)
- w = **factor de inerție** — menține direcția anterioară
 - w mare $\rightarrow$ explorare globală (pași mari)
 - w mic $\rightarrow$ exploatare locală (pași mici, convergență)
 - Poate fi constantă sau descrescătoare în timp
- c_1 = **factor cognitiv** — atracția spre propria experiență
- c_2 = **factor social** — atracția spre experiența grupului
- $r_1, r_2 \in [0, 1]$ = numere aleatoare pentru diversitate

Condiție recomandat: $c_1 + c_2 < 4$ (Carlise, 2001). Viteza e restricționată la $[-v_{max}, v_{max}]$.

16.5 Exemplu numeric PSO 2D

Minimizăm $f(x, y) = x^2 + y^2$ (minimumul în origine).

Inițializare (o particulă):

- $x = (3, 4), v = (0, 0)$
- $pBest = (3, 4), gBest = (1, 1)$ (altă particulă a găsit (1,1))
- $w = 0.7, c_1 = c_2 = 1.5, r_1 = 0.6, r_2 = 0.8$

Calculul vitezei (dimensiunea x):

$$v_x = 0.7 \cdot 0 + 1.5 \cdot 0.6 \cdot (3 - 3) + 1.5 \cdot 0.8 \cdot (1 - 3) = 0 + 0 + 1.2 \cdot (-2) = -2.4$$

Calculul vitezei (dimensiunea y):

$$v_y = 0.7 \cdot 0 + 1.5 \cdot 0.6 \cdot (4 - 4) + 1.5 \cdot 0.8 \cdot (1 - 4) = -3.6$$

Noua poziție:

$$x_{nou} = 3 + (-2.4) = 0.6$$

$$y_{nou} = 4 + (-3.6) = 0.4$$

Verificare: $f(0.6, 0.4) = 0.36 + 0.16 = 0.52 < f(3, 4) = 25 \checkmark$ — s-a îmbunătățit!

16.6 Vecinătatea în PSO

Tip vecinătate	Descriere
Globală (gBest)	Fiecare particulă cunoaște cel mai bun din TOT roiul
Locală (lBest)	Fiecare particulă cunoaște cel mai bun din vecinătatea ei
Geografică	Vecinii = particulele fizic cele mai apropiate
Socială (circulară)	Vecinii = particulele cu indici adiacenți (1-2-3-4-...-1)

Vecinătatea locală $\rightarrow$ convergență mai lentă, dar evită mai bine optimele locale.

16.7 Avantaje și limitări PSO

Avantaje	Limitări
Simplu de implementat	Poate converge prematur
Fără operatori genetici	Sensibil la parametrii w , c_1 , c_2
Performant pentru probleme continue	Nu se potrivește direct problemelor discrete
Bazat pe cooperare, nu competiție	

PARTEA a V-a — ACO

17. ACO — Ant Colony Optimization

17.1 Inspirație biologică

Furnicile naturale caută hrana lăsând **urme de feromoni** pe drumuri. Drumul mai scurt → mai multe ture → mai mult feromon → mai atrăgător → și mai multe furnici îl urmează → feedback pozitiv.

Proces:

1. Colonia de furnici pleacă în căutarea hranei
2. Apare un obstacol — furnicile îl ocolesc pe ruta A sau B
3. Ruta A e mai scurtă → furnicile fac mai multe ture → lasă mai mult feromon
4. Feromonii pe ruta B se evaporă → furnicile de pe B migrează la A
5. Toți merg pe cel mai scurt drum

17.2 Componentele ACO

Componentă	Semnificație
m	Numărul de furnici
n	Numărul de noduri/orașe
τ_{ij}	Cantitatea de feromon pe muchia (i, j)
$\eta_{ij} = 1/d_{ij}$	Euristica locală (vizibilitate) — inversul distanței
α	Importanța feromonului
β	Importanța euristicii
ρ	Rata de evaporare a feromonului ($0 < \rho < 1$)
Q	Cantitatea de feromon lăsată de o furnică
L_k	Lungimea turului efectuat de furnica k

17.3 Probabilitatea de alegere a unui pas (regula de tranziție)

O furnică aflată în nodul i alege nodul următor j cu probabilitatea:

$$P_{ij}^k = \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{l \in \text{permis}_k} \tau_{il}^\alpha \cdot \eta_{il}^\beta}$$

Unde permis_k = mulțimea orașelor pe care furnica k le mai poate vizita.

Interpretare:

- Numărătorul: atractivitatea muchiei (i, j) = feromon $\times$ euristică
- Numitorul: suma atractivităților tuturor muchiilor posibile $\rightarrow$ normalizare la probabilitate

17.4 Evaporarea feromonului

Evaporarea previne stagnarea și permite explorarea de noi rute:

$$\tau_{ij}^{(t+n)} = (1 - \rho) \cdot \tau_{ij}^{(t)} + \Delta\tau_{ij}$$

Unde:

- $(1 - \rho)$ = factor de evaporare
- $\Delta\tau_{ij}$ = cantitatea de feromon proaspăt depus

17.5 Actualizarea feromonului (Ant System — AS)

Cantitatea de feromon lăsată de furnica k pe muchia (i, j) :

$$\Delta\tau_{ij}^k = \begin{cases} Q/L_k & \text{dacă furnica } k \text{ a folosit muchia } (i, j) \\ 0 & \text{altfel} \end{cases}$$

Cantitatea totală de feromon nou:

$$\Delta\tau_{ij} = \sum_{k=1}^m \Delta\tau_{ij}^k$$

Interpretare: O furnică care a parcurs un drum scurt (L_k mic) lasă mai mult feromon (Q/L_k mare).

17.6 Algoritmul ACO pentru TSP

1. Inițializare:
 - $t = 0$
 - $\tau[i][j] = c$ (feromon inițial = constantă mică)
 - $\Delta\tau[i][j] = 0$
 - Se plasează m furnici aleator în cele n noduri
2. Cât timp nu criteriu_oprire:
 - a. Construcția soluțiilor (n pași per furnică):
 - Fiecare furnică alege următorul nod cu probabilitatea $P[i][j]$
 - Actualizează lista cu orașele vizitate
 - b. Evaluarea soluțiilor:

– Calculează L_k pentru fiecare furnică

c. Actualizarea feromonului:

- Evaporare: $\tau[i][j] = (1 - \rho) * \tau[i][j]$
- Depunere: $\tau[i][j] = \tau[i][j] + \sum \Delta\tau[i][j]^k$

$t = t + 1$

3. Returnează cel mai scurt tur găsit

17.7 Exemplu pas cu pas ACO pe TSP (4 orașe)

Graf (4 orașe A, B, C, D):

- $d(A, B) = 1, d(A, C) = \sqrt{2}, d(A, D) = 1$
- $d(B, C) = 1, d(B, D) = \sqrt{2}, d(C, D) = 1$

Parametri: $\alpha = 1, \beta = 2, \rho = 0.5, Q = 1, c = 0.5$ (feromon inițial), 2 furnici

Inițializare:

- $\tau_{ij} = 0.5$ pentru toate muchiile
- $\eta_{AB} = 1/1 = 1, \eta_{AC} = 1/\sqrt{2} \approx 0.71$, etc.

Furnica 1 pornește din A:

- Calculează $P_{AB} = \frac{0.5^{1 \cdot 1^2}}{\text{sumă}} = \frac{0.5}{0.5 \cdot 1 + 0.5 \cdot 0.71 + 0.5 \cdot 1} = \frac{0.5}{1.355} \approx 0.37$
- Similar pentru B și D
- Selectează B (cel mai probabil)

Turul final furnica 1: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A, L_1 = 1 + 1 + 1 + 1 = 4$

Actualizare feromon pe muchia AB:

$$\Delta\tau_{AB}^1 = Q/L_1 = 1/4 = 0.25$$

$$\tau_{AB}^{(t+1)} = (1 - 0.5) \cdot 0.5 + 0.25 = 0.25 + 0.25 = 0.5$$

17.8 Versiunile ACO

Versiune	Cine depune feromon	Când
AS (Ant System)	Toate furnicile	După construirea soluției complete (global colectiv)
MMAS (MaxMin Ant System)	Doar cea mai bună furnică	După construirea soluției; feromon limitat la $[\tau_{min}, \tau_{max}]$
ACS (Ant Colony System)	Toate local + cea mai bună global	Local la fiecare pas; global după soluție completă

17.9 Comparăție ACO vs AG

Caracteristică	Algoritm Genetic	ACO
Inspirație	Evoluție biologică	Comportament colectiv furnici
Soluție	Cromozom (indivd)	Tur construit de o furnică
Memorie	Populație de cromozomi	Matricea de feromoni
Comunicare	Prin selecție/crossover	Prin feromoni
Operator principal	Crossover + mutație	Depunere feromon + evaporare
Potrivit pentru	Optimizare generală	Probleme pe grafuri (TSP)

Subiect 14a din examen: "Prezentați o asemănare între AG și ACO." **Răspuns:** Ambii folosesc o **populație de soluții** care evoluează iterativ. AG lucrează cu o populație de cromozomi; ACO lucrează cu o populație de furnici care construiesc soluții în paralel. Ambii folosesc mecanisme de memorie colectivă (AG: populația; ACO: matricea de feromoni) pentru a ghida căutarea spre zone promițătoare.

PARTEA a VI-a — CRITERII DE OPRIRE

18. Criterii de Oprire în Algoritmii Evolutivi

18.1 Tipuri principale

1. **Număr maxim de generații** — cel mai simplu; $t \geq t_{max}$
2. **Număr maxim de evaluări ale funcției de fitness** — relevant când evaluarea e costisitoare
3. **Fitness satisfăcător** — $f_{best} \geq f_{target}$
4. **Convergența populației** — când diversitatea e prea mică:
 - Toți indivizii au același fitness: $f_{max} = f_{avg}$
 - Variația populației e sub un prag: $\text{var}(P) < \epsilon$
5. **Lipsa îmbunătățirii** — fitness-ul nu s-a îmbunătățit în ultimele k generații
6. **Limita de timp** — $\text{timp_rulare} \geq T_{max}$

18.2 Observație importantă

Nu există un criteriu universal. Alegerea depinde de:

- Natura problemei (discretă/continuă, unimodală/multimodală)
- Bugetul computațional disponibil
- Dacă se cunoaște soluția optimă (pentru validare)

19. ⚠️ Aspecte Critice pentru Examen

Tipare din SUBIECT_EXAMEN_2025

Subiect 3 — " k melodii, n playlists. Algoritmul genetic — care reprezentare e corectă?"

- Vrem să împărțim k melodii în n playlists cu durate egale
- Cromozomul trebuie să atribuie fiecare melodie unui playlist
- **Răspuns: d.** Un vector cu k elemente, fiecare valoare din $\{1, 2, \dots, k\} \rightarrow$ **incorect**
- **Răspuns corect: c.** Un vector cu n elemente cu valori din $\{1, 2, \dots, k\} \rightarrow$ **incorect de asemenea**

Analiză corectă: Avem k melodii care trebuie atribuite la n playlists. Cromozomul trebuie să reprezinte asignarea fiecărei melodii la un playlist. $\rightarrow$ **Vector cu k elemente, fiecare valoare din $\{1, 2, \dots, n\}$**

Subiect 14a — "Asemănare AG și ACO" $\rightarrow$ Ambii lucrează cu **populație de soluții**, folosesc **memorie colectivă**, evoluează **iterativ**

Subiect 14b — "SGD vs Batch GD" $\rightarrow$ Diferența: update după fiecare exemplu vs. după toată epoca (din Cursul 2)

Concepte-cheie de memorat

Căutare neinformată:

1. **BFS** $\rightarrow$ FIFO, completitudine Da, optimalitate Nu (față de cost), $O(b^d)$
2. **DFS** $\rightarrow$ LIFO, completitudine Nu, $O(b \cdot d_{max})$ spațiu
3. **UCS** $\rightarrow$ FIFO sortat, completitudine Da, optimalitate Da (cost minim) = Dijkstra
4. **IDS** $\rightarrow$ DFS repetat cu limită crescătoare = BFS + DFS
5. $f(n) = g(n) + h(n) \rightarrow A^*$ când h e admisibil

Căutare locală: 6. **Hill Climbing** $\rightarrow$ cel mai bun vecin, se blochează în optime locale 7. **Simulated Annealing** $\rightarrow$ acceptă soluții mai slabe cu $p = e^{\Delta E/T}$ 8. **Tabu** $\rightarrow$ HC cu listă de soluții interzise

Algoritmi evolutivi: 9. **Genotip** = reprezentarea internă (cromozom); **Fenotip** = interpretarea externă 10. **Selecție ruletă** $\rightarrow p_i = f(x_i) / \sum f(x_j)$ 11. **One-point crossover** $\rightarrow$ schimbă genele după punctul k 12. **Mutație binară** $\rightarrow x_i \leftarrow 1 - x_i$; rata mică (~ 0.01) 13. **TSP cromozom** = permutare; **fitness** = $1 / \sum d(t_i, t_{i+1})$

PSO: 14. **Formula viteză:** $v = w \cdot v + c_1 r_1 (pBest - x) + c_2 r_2 (gBest - x)$ 15. w = inerție; c_1 = cognitiv (proprie experiență); c_2 = social (grup) 16. **pBest** = cea mai bună poziție proprie; **gBest** = cea mai bună din întreg roiul

ACO: 17. **Probabilitate tranziție:** $P_{ij} = \tau_{ij}^\alpha \cdot \eta_{ij}^\beta / \sum(\dots)$ 18. **Actualizare feromon:** $\tau_{ij} = (1 - \rho)\tau_{ij} + \Delta\tau_{ij}$ 19. **Depunere feromon:** $\Delta\tau_{ij}^k = Q/L_k$ dacă furnica k a folosit muchia (i, j) 20. **AS** = toate furnicile depun; **MMAS** = doar cea mai bună; **ACS** = local + global

Rezumat generat exclusiv din [11_search_uninformed.pdf](#), [11_localSearch_EA.pdf](#), [11_localSearch_EA_suplim.pdf](#), [11_local_search_PSO_ACO.pdf](#) și fișierele .md 9_1 $\rightarrow$ 9_7 + [Criterii_de_oprire.md](#)